

Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve

1 Paper summary

This paper studies the tradeoff between throughput and latency in LLM serving. The main issue is that inference has two phases: prefill, which processes the full prompt and is compute-heavy, and decode, which generates tokens one by one and is more memory-bound. These two behave quite differently, which makes scheduling harder.

Existing systems usually optimize for one side. Some prioritize decode, which keeps token generation smooth but wastes GPU capacity, while others prioritize prefill, which improves throughput but can interrupt ongoing requests and cause delays. So in practice you don't really get both good throughput and low latency at the same time.

The paper proposes Sarathi-Serve to address this. The main idea is to split prefills into smaller chunks (called chunked-prefills), and then insert these chunks into running batches without stopping ongoing decode operations. This is called stall-free batching. It sounds simple, but it helps keep decode smooth while still using idle resources to process new requests.

Another effect of this design is that batches become more uniform, which is useful when using pipeline parallelism across multiple GPUs. It reduces idle time (pipeline bubbles), even if this depends on the setup.

The evaluation shows pretty strong improvements compared to systems like vLLM. Depending on the model, they report up to 2.6x or more improvements, and even higher gains in pipeline-parallel setups. The main takeaway is that better scheduling alone can give big benefits, not just memory optimizations.

2 Contributions

The first contribution is identifying the mismatch between prefill and decode as a key scheduling problem. The paper explains why current systems struggle to balance throughput and latency, which is useful to understand the limitations of existing approaches.

The second contribution is the chunked-prefill and stall-free batching design. By splitting prefills and inserting them into running batches, the system avoids interrupting decode requests. This helps reduce latency while still keeping GPUs busy, even if the idea requires careful implementation.

The third contribution is the improvement in pipeline-parallel scenarios. The design makes batches more uniform and reduces idle time across stages, which is important for very large models. So the benefits are not only for single-GPU setups, but also for distributed ones.

3 Problems

One limitation is that the system depends on tuning parameters like the token budget. The performance depends on choosing good values, and if the assumptions are not accurate, the balance between throughput and latency might not hold as well.

Another issue is the increased complexity of the scheduler. Splitting prefills and managing stall-free batches is more complicated than simpler approaches, and this could make implementation and debugging harder. Even if the results are good, the added complexity is not negligible.

A third point is that the evaluation is somewhat limited to specific workloads and setups. It shows good results, but doesn't fully explore things like multi-tenant environ-

ments, fairness, or highly dynamic workloads. In real systems, these aspects could have a big impact, but they are not really analyzed in detail.